

Ex.No.8 Use Steg-Expose to detect hidden data in images

Description

StegExpose is a tool used to detect hidden data in images using steganography analysis. It works by evaluating the statistical properties of an image to estimate whether any hidden data is embedded. Below is a step-by-step guide on how to use StegExpose to detect hidden data in images:

Prerequisites:

- **Java Runtime Environment (JRE):** StegExpose is a Java-based tool, so ensure you have JRE installed.
- **StegExpose Tool:** Download StegExpose from the official GitHub repository.

Step-by-Step Process:

1. Download and Set Up StegExpose:

- **Download the tool:** Go to StegExpose GitHub and download the .jar file.
- **Install Java:** Ensure that Java is installed on your machine. You can download it from the official site.
- **Prepare environment:** Place the .jar file in a folder where you plan to work on the image files.

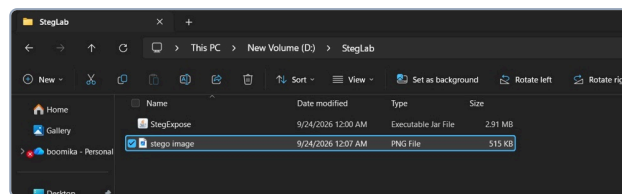


Fig 1. StegLab working directory setup

2. Select Images for Analysis:

Collect the images you suspect might have hidden data. StegExpose supports formats such as .png, .jpg, .bmp, etc.

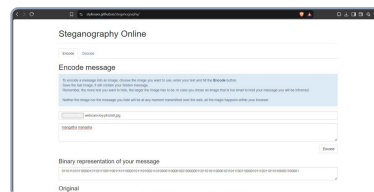


Fig 2. Generating or preparing test carrier images online

3. Open Command Line or Terminal:

- Navigate to the folder where you have saved the StegExpose.jar file.
- Open your Command Prompt (Windows) or Terminal (Linux/macOS).

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\boomi> cd D:\StegLab
PS D:\StegLab> java -jar StegExpose.jar .
stego_image.png.png is suspicious. Approximate amount of hidden data is 48093 bytes.
PS D:\StegLab> |
```

Fig 3. Navigating to the working directory in PowerShell

4. Run StegExpose on an Image:

Use the following command to check a single image for hidden data:

```
java -jar StegExpose.jar <image_file_path>
```

Example:

```
java -jar StegExpose.jar test_https://lh3.googleusercontent.com/gg/ACRwjatrCaK07qp4C-
ZQty3knn-WAifes-0fEePY00nXJcJy4kHddNPtLZY4p9-D5ReEXtp1AKZI5iys5myHBgAca2_G51nXfPztXZ-
Rtqom0_KInq8H9UGYNZLFsmBFC8df2X5tC43hMZ3UCeMnIuYllyXZoETxqddWcpFQjh09Ez5vSpFQ9XfBVHVD-
GhAHNqLeIcsBKdu6PwU9RohAX04CH0m0-
gh36iy_Z_E3k0gZxlX5n7QPtL8qnBncZjVe5YCGC4dCc7cam79Ku78_myjG641xvbKv0Qf=s0
```

Replace `<image_file_path>` with the actual path of the image you want to check.

```
PS C:\Users\boomi> cd D:\StegLab
PS D:\StegLab> java -jar StegExpose.jar .
```

Fig 4. Executing the StegExpose command

5. Analyze the Output:

After running the tool, it will analyze the image and provide a score.

Output Explanation: StegExpose calculates a "suspect" score (between 0 and 1). The higher the score, the more likely that hidden data is present.

Threshold values (suggested by the tool):

- Less than 0.2: Image is clean (no hidden data).
- 0.2 - 0.3: Possibly some hidden data.
- Above 0.3: Likely that steganography is present.

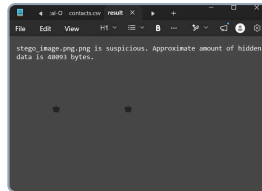


Fig 5. Analysis output showing suspicion and hidden data bytes

6. Batch Analysis (Multiple Images):

If you have multiple images to check, you can analyze them in bulk by specifying a folder. For example:

```
java -jar StegExpose.jar <folder_path>
```

Replace `<folder_path>` with the path to the folder containing the images.

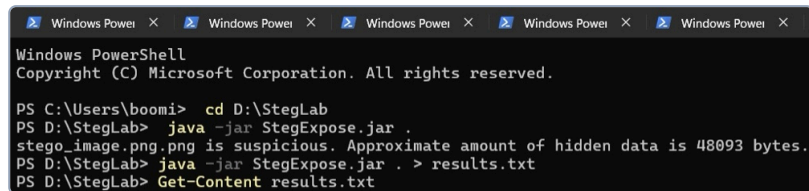


Fig 6. Batch analysis redirection to text output file

7. Advanced Options (Optional) & 8. Review Results:

Use `--help` to check options. Review scores to decide on further inspection.

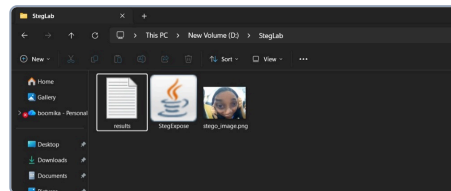


Fig 7. Results file generated inside the working directory

Example Output:

```
Analyzing suspect_https://lh3.googleusercontent.com/gg/ACRwjatrCaK07qp4C-ZQty3knn-WAifes-
0fEePY00nXJcJy4kHddNPLZY4p9-D5ReEXtp1AKZI5iym5myHBgAca2_G51nXfPztXZ-
Rtqom0_KInq8H9UGYNZLFsmBFC8df2X5tc43hM3UCeMnIuYlyyXZoETxqddWcpFQjh09Ez5vSpFQ9XfBVHvD-
GhAHNqLeIcSBKdu6PwU9RohAX04CH0m0-
gh36iy_Z_E3k0gZxlX5n7QPtL8qnBncZjVe5YCGC4dCc7cam79Ku78_myjG641xvbKv0Qf=s0...
Result: 0.4
Steganography likely present
```